



UCAM
UNIVERSIDAD CATÓLICA



UCAM
UNIVERSIDAD CATÓLICA
DE MURCIA

FINAL PROJECT

COURSE/ DOCTORATE

Computer Science/doctorate. Edition (2023-2024)

FINAL PROJECT

Optimization for Large Language Model Inference Serving: from Algorithms to Systems

STUDENT'S NAME:

Wei Wang

Abstract

Large Language Models (LLMs) have attracted extensive attention due to their remarkable performance across various tasks. However, the substantial computational and memory requirements of LLM inference pose challenges for deployment in resource-constrained scenarios. Efforts within the field have been directed towards developing techniques aimed at enhancing the efficiency of LLM inference. This paper presents a comprehensive survey of the existing literature on efficient LLM inference. We start by analyzing the primary causes of the inefficient LLM inference, i.e., the large model size, the quadratic complexity attention operation, and the auto-regressive decoding approach. Then, we introduce a comprehensive taxonomy that organizes the current literature into model-level and system-level optimization. Moreover, the paper includes comparative experiments on representative methods within critical sub-fields to provide quantitative insights. Last but not least, we provide some knowledge summary and discuss future research directions.

Keywords: Large Language Model, Inference, Efficient, Optimization

Contents

1	INTRODUCTION	5
2	PRELIMINARIES	6
2.1	Transformer-Style LLMs	6
2.2	Inference Process of LLMs	7
2.3	Efficiency Analysis	8
3	TAXONOMY	8
4	MODEL-LEVEL OPTIMIZATION	9
4.1	Efficient Structure Design	9
4.1.1	Efficient Attention Design	9
4.2	Model Compression	10
4.2.1	Quantization	10
4.2.2	Sparsification	15
5	SYSTEM-LEVEL OPTIMIZATION	18
5.1	Inference Engine	18
5.1.1	Graph and Operator Optimization	18
5.1.2	Speculative Decoding	20
5.2	Serving System	23
5.2.1	Memory Management	23
5.2.2	Continuous Batching	23
5.2.3	Distributed Systems	24
5.2.4	Parallel computation	24
6	FUTURE DIRECTION	25
7	SOFTWARE FRAMEWORKS	26
8	CONCLUSION	28

List of Figures

1.1	The challenges of LLM deployment.	5
2.1	Demonstration of the prefilling stage (a) and decoding stage (b).	7
2.2	Illustration of the memory variation through time (latency) during one generation process.	7
4.1	Overview of grouped-query method	10
4.2	Illustrations for different quantization granularity.	10
4.3	The inference workflow of Weight-only Quantization and Weight-Activation Quantization	11
4.4	Examples of different sparse attention masks.	16
5.1	Inference runtime breakdown over multiple LLMs.	18
5.2	Comparison of auto-regressive decoding (a) and speculative decoding (b).	21

List of Tables

4.1	Quantization	13
4.2	Inference frameworks for quantized large language models.	15
5.1	Speculative decoding	22
7.1	Inference framework	26

1 INTRODUCTION

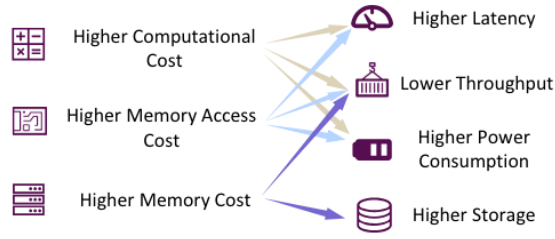


Figure 1.1: The challenges of LLM deployment.

Large Language Models (LLMs) have garnered substantial attention from both academia and industry in recent years. The field of LLMs has experienced notable growth and significant achievements. The success of LLMs stems from their robust capability in handling diverse tasks such as neural language understanding (NLU), neural language generation (NLG), reasoning, and code generation, consequently enabling impactful applications like ChatGPT, Copilot, and Bing. There is a growing belief that the rise and achievements of LLMs signify a significant stride towards Artificial General Intelligence (AGI) for humanity.

However, the deployment of LLMs is not always going smoothly. As shown in Fig. 1.1, LLMs typically demand higher computational cost, memory access cost and memory usage in their inference process (we will analyse the root causes in the Sec. 2.3), which deteriorates the efficiency indicators (e.g., latency, throughput, power consumption and storage) in the resource-constrained scenarios. This poses challenges for the application of LLMs in both edge and cloud scenarios. For example, the immense storage requirements render the deployment of a 70-billion-parameter model impractical on personal laptops for tasks such as development assistance. Additionally, the low throughput would result in significant costs if LLMs are used for every search engine request, leading to a considerable reduction in the profits of the search engine.

Fortunately, a substantial array of techniques has been proposed to enable efficient inference for LLMs. To gain a comprehensive understanding of existing studies and inspire further research, this paper employs a hierarchical classification and systematic summarization of the current landscape of efficient LLM inference. Specifically, we categorize relevant studies into 2 levels: *model-level optimization*, and *system-level optimization*. Moreover, we conduct experimental analyses on representative methods within critical sub-fields to consolidate knowledge, offer practical recommendations, and provide guidance for future research endeavors.

The remainder of this paper is organized as follows:

Sec. 2 introduces the basic concept and knowledge about LLMs and presents a detailed analysis of the efficiency bottlenecks during the inference process of LLMs.

Sec. 3 demonstrates our taxonomy.

Sec. 4-5 respectively present and discuss studies on efficiency optimization at two distinct levels.

Sec. 6 future direction

Sec. 7 software frameworks

Sec. 8 concludes the key contributions.

2 PRELIMINARIES

2.1 Transformer-Style LLMs

Transformer-based Large Language Models (LLMs) have marked a significant shift in the field of natural language processing, introducing a new paradigm for understanding and generating human language. Central to this innovation is the Transformer architecture, which is built upon the concept of self-attention mechanisms [54], allowing the model to weigh the importance of different parts of the input data when making predictions. Mathematically, the self-attention mechanism in Transformers can be described as follows: For an input sequence $X = [x_1, x_2, \dots, x_n]$, the Transformer computes a set of queries, keys, and values using linear transformations of X . The self-attention scores are then computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where d_k is the dimension of the keys. This mechanism allows the model to focus on different parts of the input sequence for each element of the output, capturing complex dependencies regardless of their distance in the input sequence.

Another important structure in Transformers is the Feed-Forward Network (FFN), which is present in each layer of the Transformer and significantly contributes to its computational intensity. The FFN typically consists of two linear transformations with a non-linear activation function in between, usually represented as:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Here, W_1 , W_2 , b_1 , and b_2 are learnable parameters of the FFN, and the non-linear function $\max(0, \cdot)$ (ReLU, in this case) introduces the necessary non-linearity into the model, allowing it to learn more complex patterns. The FFN is responsible for a significant portion of the model's parameter count and, consequently, its memory footprint and computational load. In each Transformer layer, after the multi-head attention (MHA) aggregates information from different parts of the input, the FFN processes this aggregated information independently for each position. This parallel processing capability is a key strength of the Transformer, allowing it to handle sequences effectively. However, it also means that the computational load and memory requirements scale with the length of the input sequence and the depth of the network.

The combination of self-attention and FFN in Transformer-based LLMs enables these models to capture a wide range of linguistic contexts and nuances, setting new benchmarks in various NLP tasks. However, the substantial computational requirements for training and inference have become a critical area of research, focusing on optimizing these aspects without significantly compromising performance. The Transformer model also includes other key components like position encoding, which adds information about the position of each token in the sequence, and the multi-head attention mechanism, which allows the model to focus on different parts of the sequence in different representational spaces.

2.2 Inference Process of LLMs

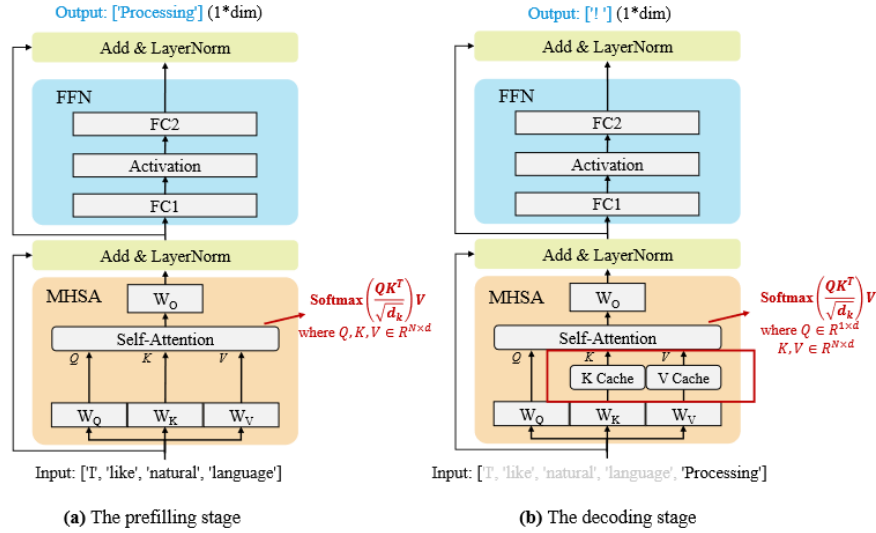


Figure 2.1: Demonstration of the prefilling stage (a) and decoding stage (b).

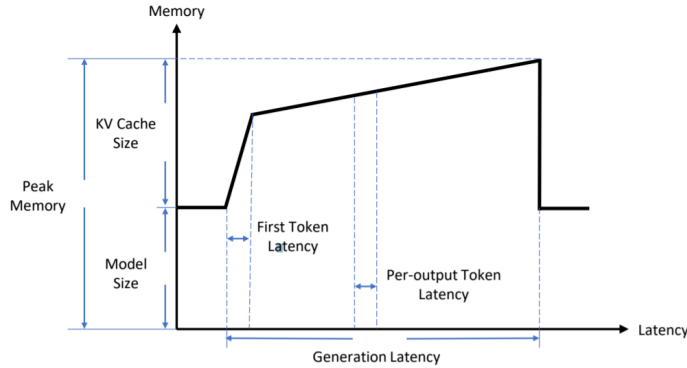


Figure 2.2: Illustration of the memory variation through time (latency) during one generation process.

The most popular LLMs, i.e., decoder-only LLMs, often adopt the auto-regressive method to generate the output sentence. Specifically, the auto-regressive method generates the tokens one by one. In each generation step, the LLM takes as input the whole token sequences, including the input tokens and previously generated tokens, and generates the next token. With the increase in sequence length, the time cost of the generation process grows rapidly. To address this challenge, a crucial technique, namely key-value (KV) cache, has been introduced to expedite the generation process.

The KV cache technique, as its name suggests, involves storing and reusing previous key (K) and value (V) pairs within the Multi-Head Self-Attention (MHSA) block. This technique has been widely adopted in LLM inference engines and systems due to its substantial optimization of generation latency. Based on the above methods and techniques, the inference process of LLMs can be divided into two stages:

- Prefilling Stage: The LLM calculates and stores the KV cache of the initial input tokens, and generates the first output token, as shown in Fig. 2.1 (a).
- Decoding Stage: The LLM generates the output tokens one by one with the KV cache, and then updates it with the key (K) and value (V) pairs of the newly generated token, as shown in Fig. 2.1(b).

As shown in Fig. 2.2, we illustrate some critical efficiency indicators. As for the latency, we denote first token latency as the latency to generate the first output token in the prefilling stage, while we denote per-output token latency as the average latency to generate one output token in the decoding stage. Besides, we use generation latency to denote the latency to generate the whole output token sequences. As for the memory, we use model size to denote the memory to store the model weights, and use KV cache size to denote the memory to store the KV cache. Additionally, peak memory denotes the maximum memory usage during the generation process, which is approximately equal to the memory sum of model weights and KV cache. Apart from the latency and memory, throughput is also a widely used indicator in the LLM serving system. We use token throughput to denote the number of generated tokens per second, and use request throughput to denote the number of completed requests per second.

2.3 Efficiency Analysis

Deploying LLMs on resource-constrained scenarios while preserving their powerful capabilities poses a significant challenge for both practitioners and researchers. In addition to storage and latency, the efficiency indicators, such as throughput, energy and power consumption, also need to be considered. During the LLM inference process, three important factors would largely affect these indicators, i.e., the computational cost, the memory access cost and the memory usage. Yuan et al. ^[62] provide a more systematic analysis to demonstrate how these factors affect the inference inefficiency with a roofline model. In the following, we further analyze three root causes of inefficiency in the LLM inference process, focusing on the above three key factors:

- Model Size: Mainstream LLMs typically incorporate billions or even trillions of parameters. For instance, the LLaMA-70B model comprises 70 billion parameters, while the GPT-3 model scales up to 175 billion parameters. This considerable model size contributes significantly to the elevated computational cost, memory access cost, and memory usage during the LLM inference process.
- Attention Operation: As illustrated in Sec. 2.1 and Sec. 2.2, in the prefilling stage, the self-attention operation exhibits quadratic computational complexity in the input length. Consequently, as the input length increases, the computational cost, memory access cost, and memory usage of the attention operation escalate rapidly.
- Decoding Approach: The auto-regressive decoding approach generates the tokens one by one. In each decoding step, all the model weights are loaded from the offchip HBM to the GPU chip, leading to a large memory access cost. In addition, the size of KV cache increases with the growth in the input length, potentially leading to fragmented memory and irregular memory access patterns.

3 TAXONOMY

In the aforementioned discussion, we identify key factors (i.e., computational cost, memory access cost and memory usage) that significantly impact the efficiency during the LLM inference process, and further analyze

three root causes (i.e., model size, attention operation and decoding approach). Many efforts have been made to optimize the inference efficiency from different perspectives. By carefully reviewing and summarizing these studies, we classify them into two levels, model-level optimization and system-level optimization

- *Model-level Optimization* refers to designing an efficient model structure (i.e., efficient structure design) or compressing the pre-trained models (i.e., model compression) in the inference process to improve its efficiency. This line of optimization (1) often requires costly pre-training or a smaller amount of fine-tuning cost to retain or recover the model ability, and (2) is typically lossy in the model performance.

- *System-level Optimization* refers to optimizing the inference engine or the serving system. This line of optimization (1) does not involve the costly model training, and (2) is typically lossless in model performance.

4 MODEL-LEVEL OPTIMIZATION

4.1 Efficient Structure Design

4.1.1 Efficient Attention Design

Multi-Query Attention. Multi-query attention (MQA) ^[47] optimizes the attention operation by sharing the key (K) and value (V) cache across different attention heads. This strategy effectively reduces both memory access cost and memory usage during inference, contributing to improved efficiency in Transformer models. As introduced in Sec. 2.2, the Transformer-style LLMs typically adopts multi-head attention (MHA) operation. This operation requires storing and retrieving K and V pairs for each attention head during the decoding stage, leading to substantial increases in memory access cost and memory usage. MQA tackles this challenge by using the same K and V pairs across different heads while maintaining distinct query (Q) values. Through extensive testing, it has been demonstrated that MQA significantly reduces memory requirements with only a minimal impact on model performance, making it a crucial strategy for enhancing inference efficiency. The concept of MQA is further extended by Grouped-query attention (GQA) ^[3], which can be seen as a blend of MHA and MQA. Specifically, GQA segments the attention heads into groups, storing a single set of K and V values for each group. This method not only sustains the benefits of MQA in reducing memory overhead but also offers an enhanced balance between inference speed and output quality.

Fig. 4.1 Multi-head attention has H query, key, and value heads. Multi-query attention shares single key and value heads across all query heads. Grouped-query attention instead shares single key and value heads for each group of query heads, interpolating between multi-head and multi-query attention.

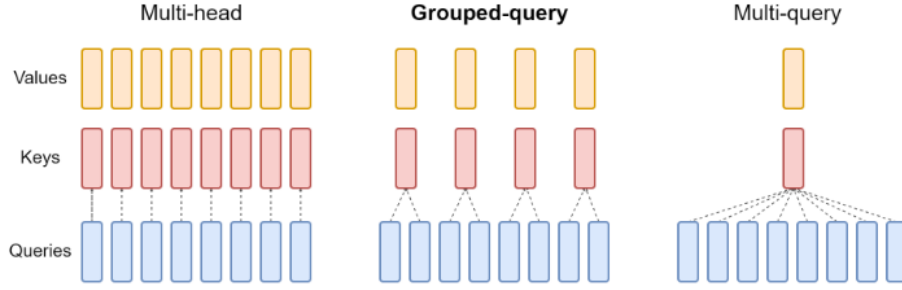


Figure 4.1: Overview of grouped-query method

4.2 Model Compression

4.2.1 Quantization

Quantization Granularity

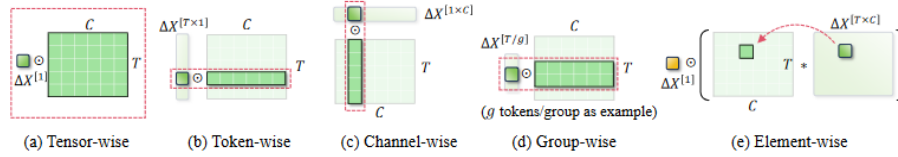


Figure 4.2: Illustrations for different quantization granularity.

Quantization granularity refers to the different weight/activation partitions corresponding to each element of the scaling factor and zeropoint. It determines how finely the scale recovers and the zero point shifts. Figure . 4.2 showcases five fundamental types of quantization granularity: tensor-wise, token-wise, channelwise, group-wise, and element-wise.

Tensor-wise is the simplest and coarsest granularity, which takes a single scaling factor and zero point to the entire tensor. It can be the fastest but may lead to the most performance degradation because it is incapable of handling the values with a wide variation. Therefore, it is unsuitable for cases where accuracy is important or the task/model is sensitive to quantization.

Token-wise is used in LLMs only, which means that each token (word or subword) has a scaling. It captures the fine-grained variations in different tokens. Usually, we adopt dynamic token-wise quantization for activation to reduce the quantization error and ensure diversity in generative models.

Channel-wise means each channel in weight within a tensor uses one scale and can be merged into quantized weight. Token-wise activation and channel-wise weight are usually used together. In this way, we preserve the generation performance with little computation overheads.

Group-wise balances the computational complexity and the quantization error by grouping tensors or channels with the same scaling factor. It also reduces the storage of scaling factors by g if there are g tokens/channels per group.

Element-wise is only applied while training the weight, which is always used together with another quantization granularity, such as tensor-wise (see Figure 2(e)). Before inference, the element-wise scaling is merged into the quantized weight. Therefore, only the tensor-wise scale is needed to be computed in inference to recover the value magnitude.

Types of Quantization

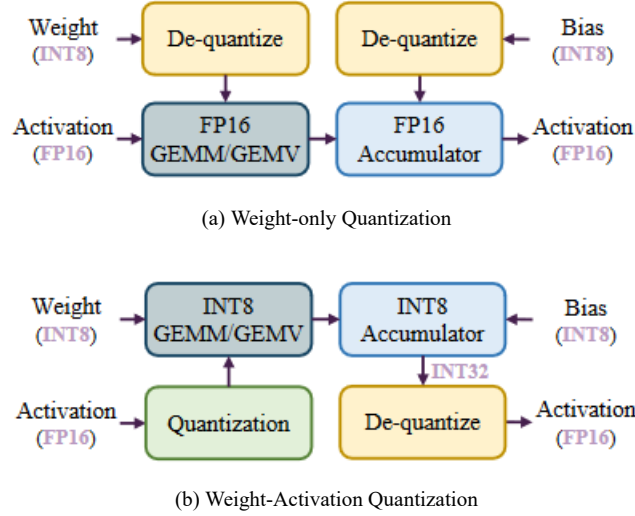


Figure 4.3: The inference workflow of Weight-only Quantization and Weight-Activation Quantization

Efficiency Analysis. As discussed in Section 2.2, the inference process of LLMs involves two stages: the prefilling stage and the decoding stage. During the prefilling stage, LLMs typically handle long token sequences, and the primary operation is general matrix multiplication (GEMM). The latency of the prefilling stage is primarily constrained by the computation performed by high-precision CUDA Cores. To address this challenge, existing methods quantize both weights and activations to accelerate computation using low-precision Tensor Cores. As illustrated in Figure . 4.3b, activation quantization is performed online before each GEMM operation, allowing computation with low-precision Tensor Cores (e.g., INT8). This quantization approach is referred to as *Weight-Activation Quantization*. In contrast, during the decoding stage, LLMs process only one token at each generation step using general matrixvector multiplication (GEMV) as the core operation. The latency of the decoding stage is mainly influenced by the loading of large weight tensors. To tackle this challenge, existing methods focus on quantizing only the weights to accelerate memory access. This method, known as *Weightonly Quantization*, involves offline quantization of weights, followed by de-quantization of the low-precision weights into FP16 format for computation, as shown in Figure . 4.3a.

It is worth noting that large models have different efficiency bottlenecks at various stages of inference, thus requiring different quantization methods to be adapted. During the prefilling Stage, the inference efficiency is mainly compute-bound, hence weight-activation quantization is commonly used. During the decoding stage, the inference efficiency is primarily memory-bound, hence weight-only quantization is typically employed.

Algorithms of quantization

This section navigates through the algorithms of LLM quantization. Quantization algorithms can be broadly divided into two primary approaches: *QuantizationAware Training (QAT)* and *Post-Training Quantization (PTQ)*. QAT integrates quantization into the training/fine-tuning process, enabling the model to learn and adapt to the quantization constraints, thereby minimizing the accuracy loss associated with lower precision. In contrast, in the scenario of PTQ, we are given a pre-trained floating-point model along with a small amount of calibration data, aiming to generate an accurate quantized model without an end-to-end training process. We will delve into these quantization algorithms in detail. By the end of this section, we hope our survey can serve as a thorough and systematic collection of the various quantization algorithms applicable to LLMs, their implementation strategies, and their implications for model performance and efficiency.

Post-Training Quantization

Post-Training Quantization (PTQ) is a technique that applies quantization to a pre-trained model. Unlike QAT, PTQ does not require the model to be trained with quantization modules. This makes PTQ a highly practical approach for deploying models that were originally trained with high precision. PTQ is particularly useful when access to the training data is limited or when retraining is computationally expensive. Therefore, with the development of the LLMs, the past few years have witnessed a remarkable surge in PTQ algorithms because of their small training cost.

In *weight-only quantization*, GPTQ^[16] represents an early advancement in LLM quantization, building upon the traditional algorithm OBQ. OBQ utilizes an optimal quantization order per row of the weight matrix, guided by the reconstruction error relative to the Hessian matrix of unquantized weights. After each quantization step, OBQ iteratively adjusts the unquantized weights to mitigate reconstruction errors. However, the frequent updating of the Hessian matrix during quantization escalates computational complexity. GPTQ streamlines this process by adopting a uniform left-to-right order for quantizing each row, thus circumventing the need for extensive Hessian matrix updates. This strategy substantially reduces computational demands by computing the Hessian matrix solely during the quantization of one row, then leveraging the computing results for subsequent rows, expediting the overall quantization procedure. LUT-GEMM presents a novel dequantization method utilizing a Look-Up Table (LUT), aiming to accelerate the inference process of quantized LLMs by reducing the dequantization overhead. Additionally, it adopts a non-uniform quantization approach known as BinaryCoding Quantization (BCQ), which incorporates learnable quantization intervals. AWQ^[31] observes that weight channels vary in importance for performance, particularly emphasizing those aligned with input channels exhibiting outliers in activations. To enhance the preservation of critical weight channels, AWQ utilizes a reparameterization method. This technique selects reparameterization coefficients via grid search to minimize reconstruction errors effectively.

For *weight-activation quantization*, LLM.int8()^[15] identifies that outliers in activations are concentrated within a small subset of channels. Leveraging this insight, LLM.int8() splits activations and weights into two distinct parts based on the outlier distribution within input channels to minimize quantization errors in activations. Channels containing outlier data in both activations and weights are stored in FP16 format, while other channels are stored in INT8 format. SmoothQuant^[58] employs a reparameterization technique to address the challenges of quantizing activation values. This method introduces a scaling factor that expands the data range

of weight channels while shrinking the data range of corresponding activation channels.

Summary

Quantization is one of the most commonly used techniques in model compression. Different quantization schemes target different levels of precision and Tensor types. For example, there are common quantization methods such as Weight Only quantization, Weight-Activation quantization, and KV Cache quantization, as shown in the following table:

Table 4.1: Quantization

Model	Quantized Tensor Type			Data Format	Quantization Parameter Determination Scheme	Quantized Value Update
	Weight	Activation	KV Cache			
GPTQ	√			Uniform	Statistic-based	√
AWQ	√			Uniform	Search-based	√
LLM.int8()	√	√		Uniform	Statistic-based	
SmoothQuant	√	√		Uniform	Statistic-based	√
KVQuant			√	Non-uniform	Statistic-based	
KIVI			√	Uniform	Statistic-based	

Performance on GPU

Weight-Only Quantization. GPTQ ^[16] is an one-shot weight quantization method based on approximate secondorder information and error compensation, that is both highly-accurate and highly-efficient. It can quantize GPT models with 175 billion parameters in approximately four GPU hours, reducing the bitwidth down to 3-bit or 4-bit per weight, with negligible accuracy degradation relative to the uncompressed baseline. Experimental results show that the average time of per token of 3-bit OPT-175B model obtained via GPTQ running on a single A100 (80GB) is 14.1 tokens/s, which is about $3.25\times$ faster than the FP16 version (running on 5 GPUs). On more accessible GPUs, such as the NVIDIA A6000 (48GB), the average time of per token is 7.7 tokens/s (running on 2 GPUs), which is about $4.53\times$ faster than the FP16 version (running on 8 GPUs). AWQ ^[31] is based on the observation that protecting 1% of salient weights whose activations are extremely large can greatly reduce quantization error. It first searches for the optimal per-channel scaling and then multiplies the salient weights with the per-channel scalings. It also reduces the bitwidth down to 3 or 4 bits per weight. Experimental results with INT4 implementation show that for Llama-2-7B, it improves the inference speed from 52 tokens/s to 194 tokens/s on RTX 4090 desktop GPU ($3.73\times$ speedup). For Llama-2-13B, the inference speed is 110 tokens/s on RTX 4090 desktop GPU. On the laptop RTX 4070 GPU (8GB), it is able to run Llama-2-13B models at 33 tokens/s, while the FP16 implementation cannot fit 7B models.

To effectively reduce the size of LLMs and preserve the model accuracy, FP6-LLM ^[57] proposes FP6 quantization on GPUs with TC-FPx, the first full-stack GPU kernel design scheme with unified Tensor Core support of float-point weights for various quantization bit-width. It solves the unfriendly memory access of model weights with irregular bit-width and high runtime overhead of weight de-quantization. Experimental results shows that for Llama2-13B with FP6 quantization, it achieves about 55 tokens/s on NVIDIA A100 GPU.

Weight-Activation Quantization. In addition to hardware units that support FP16 computations, NVIDIA GPUs also provide hardware units that support INT4, INT8, and FP8 computations. The number of these computation units can be $2\times$ and $4\times$ greater than FP16 on each chip. Compared to weight-only quantiza-

tion, weight-activation quantization can utilize INT4, INT8, and FP8 computations, thereby maximizing the peak computational performance of the GPU. Since the prefill phase in LLM inference is compute-bound, weight-activation quantization can significantly enhance performance during this stage. LLM.int8^[15] uses vector-wise quantization with separate normalization constants for each inner product in the matrix multiplication, to quantize most of the features. For the outliers, it isolates the outlier feature dimensions into a 16-bit matrix multiplication while still more than 99.9% of values are multiplied in 8-bit. For BLOOM-176B model, LLM.int8 achieves 4.05 tokens/s, 30.3 tokens/s and 109.77 tokens/s for batch size 1, 8 and 32, respectively, on 3 A100 GPUs in decode phase. The inference speed is slightly slower but close to 16-bit inference with less GPU consumption. SmoothQuant^[58] enables 8-bit weight and 8-bit activation (W8A8) quantization for LLMs. Based on the fact that weights are easy to quantize while activations are not, SmoothQuant smooths the activation outliers by offline migrating the quantization difficulty from activations to weights with a mathematically equivalent transformation. SmoothQuant enables an INT8 quantization of both weights and activations for all the matrix multiplications in LLMs. SmoothQuant achieves up to $1.56\times$ speedup and $2\times$ memory reduction for LLMs with negligible loss in accuracy.

Compared to integer quantization, floating-point (FP) quantization can better handle long-tail or bell-shaped distributions, and it has emerged as a default choice in many hardware platforms. LLM-FP4^[34] quantizes both weights and activations in LLMs down to 4-bit floating-point values (W4A4) with negligible accuracy loss. Due to the lack of PF4 computing unit in GPUs, its decoding speed maybe slower than FP16 baseline.

Inference Framework for Quantization

With the emergence of quantization algorithms for LLMs, some typical methods have already been integrated into most frameworks, while some methods may be developed and published originally on a specific framework. We list the most ready-to-use algorithms in each mainstream framework in Table 4.2. Some methods are included by the most frameworks, such as GPTQ^[16], AWQ^[31], SmoothQuant^[58] and so on. These methods share several advantages: high accuracy and efficient performance after quantization, seamless integration into existing implementation procedures, and user-friendliness.

In addition, some algorithms are supported by several frameworks. For example, LLM.int8()^[15] was well supported by bitsandbytes (by HuggingFace), which allows to store and load 8-bit weights directly from the HuggingFace Hub and quantize weight in linear layers to 8-bit. FP6-LLM^[57] is integrated in DeepSpeed-FastGen to implement the runtime quantization for 6-bit floating-point weight-only quantization. It allows efficient quantization and dequantization of 6-bit weight LLMs through a unified configuration option. It is noteworthy that Transformers (by HuggingFace) integrate most algorithms with comprehensive user manuals and detailed examples, enabling a quick start for deep learning researchers and developers.

Table 4.2: Inference frameworks for quantized large language models.

Framework	Algorithms	Model family	Device	Organ.
TensorRT-LLM	AWQ, GPTQ, LoRA, SmoothQuant	Transformer-like (Llama)	NVIDIA GPU, Intel CPU	NVIDIA
Transformers	AQLM, AWQ, AutoGPTQ, EETQ, HQQ, Exllama, QAT, Quanto, SmoothQuant	Mixture-of-Expert, Transformer-like (Llama)	Intel CPU, MPS, Numba, NVIDIA GPU, Triton	HuggingFace
OpenVINO	AWQ, GPTQ, SmoothQuant	Multi-modal	CPU, NPU, NVIDIA GPU	Intel
bitsandbytes	LLM.int8(), LoRA	Mixture-of-Expert, Transformer-like (Llama)	AMD GPU, Intel CPU	HuggingFace
vLLM	AQLM, AWQ, GPTQ, LoRA, SmoothQuant, SqueezeLLM	Mixture-of-Expert, Multi-modal LLMs (e.g. LLaVA), Transformer-like (Llama)	AMD GPU, Neuron devices, NVIDIA GPU, TPU, XPU, x86 64 CPU	University of California, Berkeley
LMDeploy	AWQ	Mixture-of-Expert, Multi-modal LLMs (e.g. LLaVA), Transformer-like (Llama), falcon	Jetson, NVIDIA GPU	Shanghai AI Lab & SenseTime
SGLang	AWQ, GPTQ	Mixture-of-Expert, Transformer-like (Llama), Multi-modal	NVIDIA GPU	LMSYS.org

4.2.2 Sparsification

Sparsification is a compression technique that increases the proportion of zero-valued elements in data structures such as model parameters or activations. This method aims to decrease computational complexity and memory usage by efficiently ignoring zero elements during computation. In the context of LLMs, sparsification is commonly applied to weight parameters and attention activations. It leads to the development of *sparse attention mechanisms*.

Sparse Attention

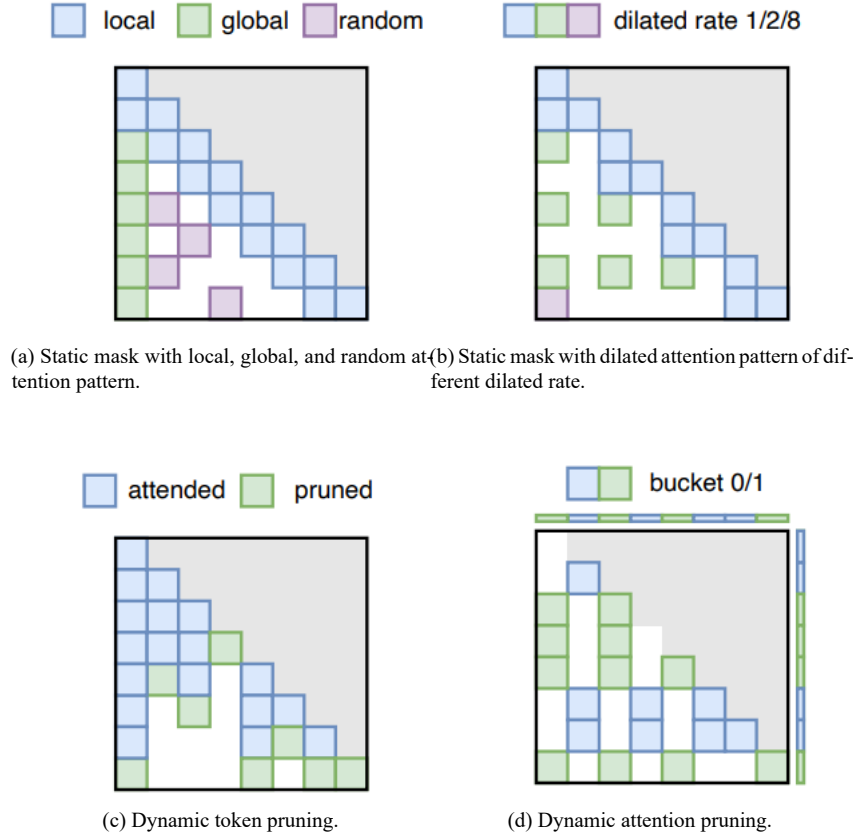


Figure 4.4: Examples of different sparse attention masks.

Static sparse attention removes activation values independently of specific inputs [9], [6], [63]. These methods pre-determine the sparse attention mask and enforce it on the attention matrix during inference. Previous studies combine different sparse patterns to preserve the most essential elements within each attention matrix. As shown in Figure 4.4a, the most common sparse attention patterns are the local and global attention patterns. The local attention pattern captures the local context of each token with a fixed-size window attention surrounding each token. The global attention pattern captures the correlation of specific tokens to all other tokens by computing and attending to all tokens across the sequence. Note that leveraging global patterns can eliminate the need to store key-value (KV) pairs for unused tokens, thereby reducing memory access cost and memory usage during the decoding stage. Sparse Transformer [9] combines these patterns to capture the local context with a local pattern, and then aggregates the information with the global pattern for every few words. StreamingLLM [59] applies the local pattern, along with the global pattern only for the first few tokens. It shows that such a global pattern serves as the attention sink to keep the strong attention scores toward initial tokens. It helps the LLMs to generalize to infinite input sequence length. Bigbird [63] also uses the random pattern, where all tokens attend to a set of random tokens. The combination of local, global and random patterns is proven to encapsulate all continuous sequence-to-sequence functions, affirming its Turing completeness. As shown in Figure 4.4b, Longformer [6] additionally introduces the dilated sliding window

pattern. It is analogous to dilated CNNs and makes the sliding window “dilated” to increase the receptive field.

In contrast, *Dynamic sparse attention* adaptively eliminates activation values based on varying inputs, employing real-time monitoring of neuronal activation values to bypass computations for neurons with negligible impact, thereby achieving pruning. Most dynamic sparse attention methods employ the dynamic token-pruning methods, as Figure 4.4c shows. Adaptively Sparse Attention^[5] leverage the inherent redundancy in linguistic constructs to propose dynamic tokenlevel pruning strategies. Adaptively Sparse Attention^[5] gradually prunes the tokens during the generation process. It drops parts of the context that are no longer required for future generation.

In addition to dynamic token pruning, dynamic attention pruning strategies are also employed^[24],^[46],^[66]. As Figure 4.4d shows, instead of pruning all the attention values of certain tokens, these methods dynamically prune the selective part of the attention based on the input. A prominent approach within this domain is dynamically segmenting input tokens into groups, known as buckets, and strategically omitting the attention calculations for tokens that reside in separate buckets. The challenge and focus of these methods lie in the way to cluster related tokens together, thereby facilitating attention computations solely among them to enhance efficiency. Reformer^[24] leverages locality-sensitive hashing to cluster keys and queries that share identical hash codes into the same bucket. Meanwhile, the Routing Transformer^[46] employs a spherical k-means clustering algorithm to aggregate tokens into buckets, optimizing the selection process for attention computations. Diverging from the bucket-level operation, H2O^[66] introduces the tokenlevel dynamic attention pruning mechanism. It combines static local attention with dynamic computations between the current query and a set of dynamically identified key tokens, termed heavy-hitters (H2). These heavy-hitters are dynamically adjusted with an eviction policy aimed at removing the least significant keys at each generation step, effectively managing the size and relevance of the heavyhitter set.

Performance on GPU

Attention Sparsity. During the prefilling phase of LLM inference, attention computation complexity scales quadratically with input sequence length. Given limited GPU computing and memory resources, attention sparsification can reduce the number of attention values to accelerate prefilling phase. For static sparsity, Sparse Transformer^[9], StreamingLLM^[59], Bigbird^[63], and Longformer^[6] use the manual combination of global and local patterns to replace the full attention patterns. The local pattern captures the local context of each token within a fixed size or stride while the global pattern captures the relationship between the specific tokens to all other tokens. For Llama2-7B and Llama2-13B models, StreamingLLM achieves 15.38-32.26 tokens/s and 9.43-20.83 tokens/s on single NVIDIA A6000 GPU, respectively. For dynamic sparsity, Adaptively Sparse Attention^[5] replaces softmax with α -entmax, a differentiable generalization of softmax that allows low-scoring words to receive precisely zero weight and drops parts of the context that are no longer required for future generation. Reformer^[24] replaces dot-product attention by using locality-sensitive hashing, changing the complexity from $O(L^2)$ to $O(L \log L)$, where L is the sequence length. H2O^[66] observes that a small portion of tokens (called Heavy Hitters, H2) contributes most of the value when computing attention scores. H2O introduces a dynamic attention sparsification method to adopt KV cache eviction policy that dynamically retains a balance of recent and H2 tokens. For OPT-6.7B model, H2O with 20% H2 achieves 30.4 tokens/s on single NVIDIA T4 GPU.

5 SYSTEM-LEVEL OPTIMIZATION

5.1 Inference Engine

5.1.1 Graph and Operator Optimization

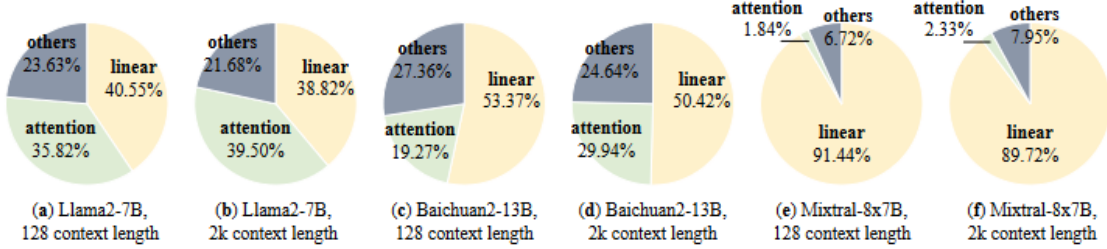


Figure 5.1: Inference runtime breakdown over multiple LLMs.

Attention Operator Optimization. The standard attention computation (e.g., using Pytorch) involves the multiplication of the Query matrix (Q) with the Key matrix (K), resulting in quadratic time and space complexity in relation to the input sequence length. As shown in Fig. 5.1, the time proportion of the attention operator increases as the context length grows. This translates to high demands on memory size and computational capability, especially when dealing with long sequences. To address the computational and memory overhead of standard attention computation on GPUs, customized attention operators are essential. FlashAttention^{[12], [11]} fuses the entire attention operation into a single, memory-efficient operator to alleviate memory access overhead. The input matrices (Q, K, V) and attention matrix are tiled into multiple blocks, which eliminates the need for complete data loading. Built upon Flash Attention, FlashDecoding^[13] aims to maximize computational parallelism for decoding. Due to the application of the decoding approach, the Q matrix degrades into a batch of vectors during decoding, which makes it challenging to fill the computational units if the parallelism is limited to the batch size dimension. FlashDecoding addresses this by introducing parallel computation along the sequence dimension. While this introduces some synchronization overhead to softmax computation, it leads to noticeable improvements in parallelism, particularly for small batch sizes and long sequences. The subsequent work, FlashDecoding++^[21], observes that in previous works^{[12], [11], [13]}, the maximum value within the softmax only serves as a scaling factor to prevent data overflow. However, the dynamical maximum value incurs significant synchronization overhead. Moreover, extensive experiments indicate that in typical LLM (e.g., Llama2, ChatGLM), over 99.99% of the softmax inputs fall within a certain range. Thus, FlashDecoding++ proposes to determine the scaling factor based on statistics in advance. This eliminates the synchronization overhead in softmax computation, enabling parallel execution of subsequent operations alongside the softmax computation.

Linear Operator Optimization The linear operator plays a pivotal role in LLM inference, performing in feature projection and Feedforward Neural Networks (FFNs). In traditional neural networks, linear operators can be abstracted into General Matrix-Matrix Multiplication (GEMM) operations. However, in the case of LLM, the application of the decoding approach results in a notably reduced dimension, diverging from the conventional GEMM workload. The low-level implementation of traditional GEMM has been highly opti-

mized, and mainstream LLM frameworks (e.g., DeepSpeed^[45], vLLM^[25] and etc.) primarily call the GEMM APIs offered by cuBLAS^[40] for linear operators.

Without an explicitly tailored implementation for GEMMs with a reduced dimension, the linear operators during decoding suffer inefficiency. A notable trend to address the issue is observed in the latest release of TensorRT-LLM^[53]. It introduces a dedicated General Matrix-Vector Multiplication (GEMV) implementation, potentially improving efficiency for the decoding step. Recent research FlashDecoding++^[21] makes a further step, addressing the inefficiency of cuBLAS^[40] and CUTLASS^[41] libraries when dealing with small batch sizes during the decode step. The authors first introduce the concept of the FlatGEMM operation to represent the workload of GEMM with a highly reduced dimension (dimension size < 8 in FlashDecoding++). As FlatGEMM poses new computational characteristics, the tiling strategy for traditional GEMMs necessitates modification to be applied. The authors observe that two challenges exist as the workload varies: low parallelism and memory access bottleneck.

To tackle the challenges, FlashDecoding++ adopts a finegrained tiling strategy to improve parallelism, and leverages the double buffering technique to hide memory access latency. Furthermore, recognizing that the linear operations in typical LLM (e.g., Llama2, ChatGLM) often have fixed shapes, FlashDecoding++ establishes a heuristic selection mechanism. This mechanism dynamically chooses between different linear operators based on the input size. The options include FastGEMV, FlatGEMM, and GEMM provided by cuBLAS^[40],^[41] libraries. This approach ensures the selection of the most efficient operator for the given linear workload, potentially leading to better end-to-end performance.

Performance on GPU

Fusion. To address the quadratic memory requirements in the attention computation, FlashAttention^{[12][11]} fuses the attention operation into a single operator by tiling input matrices (Q, K, V) and the attention matrix into blocks. Based on FlashAttention, FlashDecoding^[13] proposes additionally the parallel computation along the feature dimension, improving performance for small batch size during the decoding phase. FlashDecoding achieves 95.07 tokens/s and 54.19 tokens/s for Llama2-7B on a single NVIDIA A100 and RTX 3090 GPU respectively. Subsequently, FlashDecoding++^[21] optimizes the synchronization overhead in softmax computation by pre-determining a unified maximum based on statistical analysis in advance, enabling the parallel execution of subsequent operations and improving efficiency in typical LLMs like Llama2 and ChatGLM. FlashDecoding++ achieves 115.57 tokens/s and 61.66 tokens/s for Llama2-7B on a single NVIDIA A100 and RTX 3090 GPU respectively.

The linear operator is widely used in deep learning. In the common framework for deep learning (e.g., Pytorch), the backend of the linear operator is usually the General Matrix-Matrix Multiplication (GEMM) operation supported by NVIDIA. The naive implementation by HuggingFace achieves 44.60 tokens/s and 36.29 tokens/s for Llama2-7B on a single NVIDIA A100 and RTX 3090 GPU respectively. In the LLM framework (e.g., DeepSpeed^[45], vLLM^[25], and OpenPPL^[175]), the GEMM implementation provided by cuBLAS^[40] APIs is generally optimized. For the Llama2-7B model on a single NVIDIA A100 GPU, they achieve 89.24 tokens/s, 88.45 tokens/s and 93.63 tokens/s. For the Llama2-7B model on a single NVIDIA RTX 3090 GPU, they achieve 51.28 tokens/s, 53.31 tokens/s and 55.99 tokens/s. To address the inefficiency of GEMM during decoding due to the reduced dimensions, TensorRT-LLM^[53] introduces a dedicated General Matrix-Vector Multiplication (GEMV) implementation to support the decoding phase of LLM when batch

size equals 1. TensorRT-LLM achieves 98.19 tokens/s for Llama2-7B on a single NVIDIA A100 GPU. For the smaller batch size during the decoding phase, FlashDecoding++^[21] introduces FlatGEMM to address the inefficiencies in cuBLAS^[40] and CUTLASS^[41] libraries and employs fine-grained tiling and double buffering techniques to improve parallelism and reduce the latency of memory access. Moreover, it adopts a heuristic selection mechanism to dynamically select the most efficient operator based on the input. The performance of FlashDecoding++ is shown above.

Operator fusion^{[12][64][45][21]} is a common and effective optimization to reduce the runtime memory access, eliminate kernel launching overhead and enhance parallelism. ByteTransformer^[64] and DeepSpeed^[45] fuse the main operator (e.g., GEMM) and the lightweight operators (e.g., residual adding, layernorm and activation functions) into a single kernel to reduce the kernel launching overhead. FlashAttention^[12] mentioned above fuse the attention operator into one single kernel, reducing significantly the overhead of memory access and the memory requirements. FlashDecoding++^[21] also achieves the integration of seven fused kernels in a transformer block.

5.1.2 Speculative Decoding

Speculative decoding^{[65], [56]} is an innovative decoding technique for auto-regressive LLMs designed to enhance decoding efficiency without compromising the fidelity of outputs. The core idea of this approach involves employing a smaller model, termed a draft model, to predict several subsequent tokens efficiently, followed by validation of these predictions using the target LLM in parallel. This methodology aims to enable the LLM to generate multiple tokens within the time frame typically required for a single inference.

Fig. 5.2 demonstrates the comparison of the traditional auto-regressive decoding method and the speculative decoding approach. Formally, speculative decoding approach consists of two steps:

- 1) Draft Construction: It employs the draft model to generate several subsequent tokens, namely draft tokens, in parallel or in the auto-regressive manner.
- 2) Draft Verification: It employs the target model to compute the conditional probabilities of all the draft tokens in a single LLM inference step, subsequently determining the acceptance of each draft token sequentially. The acceptance rate, representing the average number of accepted draft tokens per inference step, serves as a key metric for evaluating the performance of a speculative decoding algorithm.

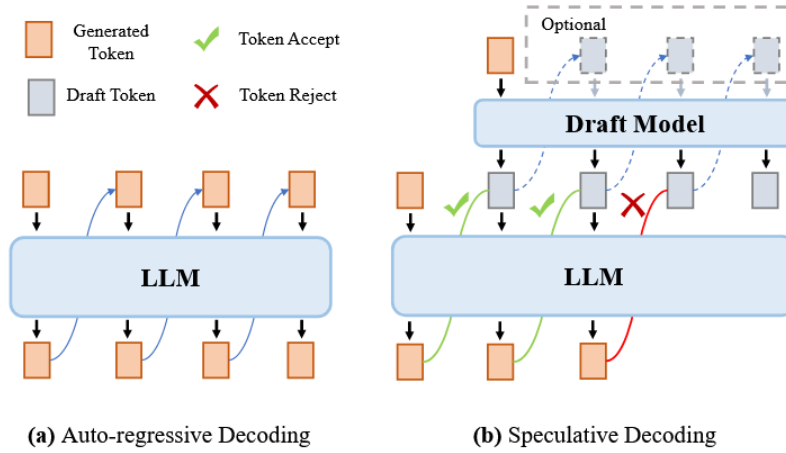


Figure 5.2: Comparison of auto-regressive decoding (a) and speculative decoding (b).

Draft Verification Speculative decoding ensures output equivalence with standard auto-regressive decoding methods. Traditional decoding techniques typically employ two primary sampling strategies: *greedy sampling* and *nucleus sampling*. Greedy sampling involves selecting the token with the highest probability at each decoding step to generate a specific output sequence. The initial attempt at speculative decoding, known as Blockwise Parallel Decoding [50], aims to ensure that the draft tokens precisely match the tokens sampled via greedy sampling, thus preserving output token equivalence. In contrast, nucleus sampling involves sampling tokens from a probability distribution, resulting in diverse token sequences with each run. This diversity makes nucleus sampling popular. To accommodate nucleus sampling within speculative decoding frameworks, speculative sampling techniques [26], [8] have been proposed. Speculative sampling maintains output distribution equivalence, aligning with the probabilistic nature of nucleus sampling to generate varied token sequences.

Building upon speculative sampling, several variants [36], [52] have emerged, aimed at validating multiple draft token sequences. Notably, the token tree verifier [36] has become a widely adopted verification strategy within this context. This approach utilizes a tree-structured representation of draft token sets and employs a tree attention mechanism to efficiently perform the verification process.

Draft Construction In the speculative decoding approach, the acceptance rate of draft tokens is significantly influenced by the degree to which the output distributions of draft models align with those of original LLMs. As a result, considerable research efforts have been directed towards improving *the design of draft models*. DistillSpec [69] directly distills a smaller draft model from the target LLM. REST [19] introduces a retrieval-based speculative decoding approach, employing a non-parametric retrieval data store as the draft model. SpecInfer [36] introduces a collective boost-tuning technique to align the output distribution of a group of draft models with that of the target LLM. Lookahead decoding [17] involves generating n-grams of the target LLM in parallel to aid in generating draft tokens. Medusa [7] finetunes several heads of the LLM specifically for generating subsequent draft tokens. Eagle [28] adopts a lightweight transformer layer called an auto-regression head to generate draft tokens in an auto-regressive manner, integrating rich contextual features from the target LLM into the draft model’s input.

Summary

We summarize the main features of existing Speculative Decoding methods in Table 5.1, including the drafter type or the drafting strategy, supported verification strategies, and the reported speedup, etc. In Table 5.1, “Independent-D” and “Self-D” denote independent drafting and self-drafting, respectively. “Greedy”, “Sampling”, and “Token Tree” denote whether the method supports greedy decoding, speculative sampling, and token tree verification, respectively. We list the most representative target LLMs for each method and the speedups in the original paper (if reported), which is obtained with a batch size of 1.

Table 5.1: Speculative decoding

Methods		Drafting		Verification			Token Acceptance Rate	Speedup (reported)
		Approach	Tuning-free	Greedy	Sampling	Token Tree		
Independent-D	DistillSpec	Small LM	N	√	√		-	-
	SpecInfer	Boost-tuned LMs	N	√	√	√	20%	2.0x-2.4x
	REST	Context Retrieval	Y	√	√	√	<10%	1.6x-1.8x
Self-D	Blockwise	FFN Heads	N	√			60%	1.7x-3.0x
	Medusa	FFN Heads	N	√	√	√	<10%	2.2x-2.3x
	Lookahead	Mask-P & N-grams	Y	√			<3%	1.5x-2.3x
	EAGLE	Auto-regression Head	N	√	√	√	40%-75%	2.9x-3.1x

Performance on GPU

Speculative Decoding. Speculative decoding^[50] is proposed to overcome the inherently sequential process in the autoregressive decoding of LLM. The essential decoding mechanism is to make predictions (i.e., draft tokens) parallelly for multiple time steps and then select the longest prefix verified by a scoring model as the final output. Lookahead decoding^[17] adopts the Guess-and-Verify paradigm as the whole decoding mechanism which generates the draft tokens by n-gram method and verifies the draft tokens during the forward at the same time. For Llama-7B and Llama-13B models with different datasets, it achieves 65.12-94.51 tokens/s and 56.01-90.05 tokens/s on a single NVIDIA A100-80GB GPU. To improve the acceptance rate while maintaining generation quality, many works focus on the scheme of generating draft tokens. Medusa^[7] adds extra decoding heads to generate multiple subsequent tokens in parallel and uses a tree-based attention mechanism to construct multiple candidate continuations and verify them simultaneously in each decoding step. For Vicuna-7B and Vicuna-13B, it achieves 129.86 tokens/s and 98.54 tokens/s on a single NVIDIA A100-80GB GPU. EAGLE^{[28][27]} selects a single transformer layer with the same configuration as LLM as the draft model to make predictions autoregressively and combines the feature and token embedding as the input of the draft model. During the verification phase on the target model, EAGLE chooses the tree-based attention mechanism similar to Lookahead and Medusa to ensure the correct relationship between the draft tokens. For Vicuna-7B, Vicuna-13B, Llama2-Chat-7B, and Llama-2-Chat-13B models, it achieves 139.95 tokens/s, 132.31 tokens/s, 133.98 tokens/s, and 156.80 tokens/s on a single NVIDIA A100-80GB GPU. Based on the basic paradigm of speculative decoding for prediction by draft models and verification by target models, some studies explore the optimizations on the modules in it.

5.2 Serving System

5.2.1 Memory Management

The storage of KV cache dominates the memory usage in LLM serving, especially when the context length is long (see Sec. 2.3). Since the generation length is uncertain, it is challenging to allocate the space for KV cache storage in advance.

the static way of KV cache memory allocation still fails when no such large contiguous space exists. To deal with the fragmented storage, vLLM^[25] proposes to store the KV cache in a paged manner following the operating system. vLLM first allocates a memory space as large as possible and divides it equally into multiple physical blocks. When a request comes, vLLM dynamically maps the generated KV cache to the pre-allocated physical blocks in a discontinuous fashion. In this way, vLLM significantly reduces storage fragmentation and achieves a higher throughput in LLM serving. On the basis of vLLM, LightLLM^[39] uses a more fine-grained KV cache storage to cut down the waste happening with the irregular boundary. Instead of a block, LightLLM treats the KV cache of a token as a unit, so that the generated KV cache always saturates the pre-allocated space.

Current optimized service systems commonly employ this paged approach to manage the KV cache storage, thereby mitigating the waste of redundant KV cache memory. However, the paged storage leads to irregular memory access in the attention operator. For the attention operator using the paged KV cache, this necessitates the consideration of the mapping relationship between the virtual address space of the KV cache and its corresponding physical address space. To enhance the efficiency of the attention operator, the loading pattern of the KV cache must be tailored to facilitate contiguous memory access. For instance, in the case of the PagedAttention by vLLM^[25], the storage of the head size dimension is structured as a 16-byte contiguous vector for K cache. The optimization of the attention operator in conjunction with paged KV cache storage remains a forefront challenge in the advancement of serving systems.

5.2.2 Continuous Batching

The request lengths in a batch can be different, leading to low utilization when shorter requests are finished and longer requests are still running. Due to the asynchronous nature of requests in serving scenarios, there exists an opportunity that such periods of low utilization could be mitigated. The continuous batching technique is proposed to leverage the opportunity by batching new requests once some old requests are finished. ORCA^[61] is the first to utilize the continuous batching technique in LLM serving.

The computation of each request encompasses multiple iterations, with each iteration representing either a prefilling step or a decoding step. The author suggests that different requests can be batched at the iteration level. The work implements iteration-level batching in linear operators, concatenating different requests together in the sequence dimension. Hence, the spare storage and computational resources corresponding to the completed requests are promptly released. Following ORCA, vLLM^[25] extends the technique to the attention computation, enabling requests with different KV cache lengths to be batched together. Sarathi^[2], DeepSpeed-FastGen^[20] and SarathiServe^[1] further introduce a split-and-fuse method to batch together prefilling requests and decoding requests. Specifically, this method first splits the long prefilling request in the sequence dimension, and then batches it together with multiple short decoding requests. The split-and-fuse method balances the workloads among different iterations, and significantly reduces the tail latency via

removing the stalls from new requests. LightLLM^[39] also adopts the split-and-fuse method.

The split-and-fuse technology operates on the premise that requests during the prefilling stage can be partitioned into discrete chunks. Chunked-prefill methodology involves segmenting prefilling requests along the sequence dimension, thereby preventing the potential bottlenecks for other requests. This strategy capitalizes on the auto-regressive characteristics inherent in LLMs, where attention computation only relies on prior tokens. Consequently, the mathematical equivalence of chunked-prefill technology is guaranteed, positioning it as a leading approach for reducing request latency in LLM serving.

5.2.3 Distributed Systems

In order to achieve high throughput, LLM services are commonly deployed on distributed platforms. Recent works have additionally focused on optimizing the performance of such inference services by exploiting distributed characteristics. Notably, observing that the computations of prefilling and decoding have interference with each other, splitwise^[43], TetriInfer^[22] and DistServe^[68] demonstrate the efficiency of disaggregating the prefilling and the decoding steps of a request. In this way, the two distinct steps are processed independently based on their characteristics. Llumnix^[51] reschedules the requests at runtime for different serving objectives including load balancing, de-fragmentation, and prioritization. SpotServe^[37] is designed to provide LLM service on clouds with preemptible GPU instances. SpotServe efficiently handles challenges including dynamic parallel control and instance migration, and also utilizes the auto-regressive nature of LLMs to achieve token-level state recovery. Moreover, Infinite-LLM^[30] parallels different parts of the sequence in the attention operator across the data center, to address the challenges when serving extremely long contexts. LoongServe^[55] proposes the elastic sequence parallelism to manage the elastic resource demand at the iteration level, reducing the data movement of KV cache via elaborately designed scheduling.

5.2.4 Parallel computation

Model parallelism Most model parallelism approaches are first proposed for distributed training of large-scale DNNs, especially for Transformer-based models. For example, tensor model parallelism^[49] (TP) splits the model layers (e.g., attention, FFN) into multiple pieces from internal dimensions (e.g., head, hidden) and deploys each on a separate device (e.g., GPU). It can significantly reduce inference latency through parallel computing, which is widely used for multiple GPUs within the same machine, especially for scenarios with high-speed NVLink connections. PaLM inference^[44] extends TP on large-scale Transformer inference by involving 2D tensor parallelism and claims lower theoretical communication complexity for large clusters (more than 256 devices). For multi-query attention with only one head for keys and values, it further involves data parallelism to the hybrid tensor partition strategy. Pipeline model parallelism (PP) arranges the model layers in a sequence across multiple devices. Each device is responsible for a pipeline stage that consists of multiple consecutive model layers. While PP can significantly increase the number of inputs processed per unit of time (throughput), it doesn't inherently decrease the time taken to process a single input from beginning to the end (latency) like TP. Sequence parallelism (SP) has various differentiated designs and implementations, but its key idea for LLM inference is to distribute the computational and storage load by splitting the processing of long sequences across multiple GPUs along the sequence length dimension^[32]. Different parallelism techniques introduce varying degrees of communication overhead and computational latency. To achieve optimal

performance and resource utilization, automatic parallelism has been widely studied by prior approaches for distributed training (e.g., Alpa [67], Galvatron [38]). By replacing their cost model to fit the predictable runtime of auto-regressive inference of Transformer models like, it’s easy to apply previous automatic searching algorithms (e.g., dynamic programming, integer linear programming) to LLM serving (e.g., AlpaServe [29], FlexFlow-Serve [10], SpotServe [37]) and determine the most efficient parallelism strategy without manual intervention. There are also some approaches [4][36][48] enabling offloading techniques to use larger but slower memory (e.g., CPU DRAM) to save model parameters and KV cache in addition to the limited device memory (e.g., GPU DRAM).

6 FUTURE DIRECTION

As we stand at the forefront of LLM advancements, it becomes increasingly important to not only understand the current state of these technologies but also to anticipate and shape their future trajectory. Particularly in the realm of generative LLM serving, there is a vast landscape of unexplored possibilities and emerging challenges. The rapid evolution of this field necessitates a forward-looking approach, where identifying potential avenues for innovation and improvement is crucial. This foresight not only prepares us to adapt to upcoming technological shifts but also guides the research community toward addressing the most pertinent and impactful areas. In this context, we outline several promising directions for future research and development, each offering the potential to significantly enhance the efficiency of serving generative LLMs.

• Development and Enhancement of Hardware Accelerators.

Future progress in enhancing generative LLM serving efficiency could be significantly driven by the development and refinement of specialized hardware accelerators, complemented by a co-design approach that aligns hardware and software optimizations. For instance, integrating memory closer to processing units or optimizing chip architectures to better align with the data flow of LLM algorithms can lead to substantial reductions in latency and energy consumption. This approach has been exemplified in recent GPU advancements, like NVIDIA’s Hopper architecture [42], which demonstrates improvements in HBM and SRAM capacity, memory bandwidth, computing units and bisection bandwidth, directly benefiting the processing of LLMs. Continued innovation in this area could involve designing hardware that is inherently tuned to the computational patterns of generative LLMs, such as optimizing for the specific demands of attention mechanisms and tensor operations that are prevalent in these models, eventually influencing the design and implementation of LLM serving systems.

• Efficient and Effective Decoding Algorithms.

The development of more efficient decoding algorithms could substantially improve serving efficiency. Motivated by the demand for more resource-efficient ways to utilize the vast knowledge encapsulated within LLMs, future work could explore alternative approaches to the traditional auto-regressive methods and unlock the generation speed for real-time applications while maintaining the decoding quality. One promising direction is generalized speculative inference as it enables preserving the same generation quality. Specifically, the small speculative model can be generalized to any other forms of methods that can generate draft tokens more efficiently than LLMs, such as knowledge retriever and user-defined functions [36]. For example, some subsequent works arose recently, replacing the draft model with early exiting or non-autoregressive decoding [17]. In summary, the development of efficient decoding algorithms like speculative decoding coupled with

the underlying system optimizations represents a significant opportunity to enhance the serving efficiency of generative LLMs.

• **Long Context/Sequence Scenarios Optimization.**

As the application of LLMs continues to expand into more sophisticated scenarios, the demand for processing longer contexts or sequences is steadily growing. Serving LLMs with long-sequence workloads requires resolving the challenges from both the algorithm and system sides. In terms of LLMs, they often suffer from length generalization failure when sequences get longer than what was observed during training even enabling relative positional encoding or after fine-tuning on longer corpora. Even for some models that claim to support ultra-long contexts, studies have found that they encounter a situation of “loss in the middle” [33]. Current approaches attempt to alleviate such limitations by reducing the computational sequence length while preserving relevant information, such as retrieval augmentation [60], sequence compression [23] and caching [18]. For the LLM serving systems, longer sequence brings critical challenges, including more memory consumption and access of KV cache and quadratic increasing computational complexity of self-attention.

7 SOFTWARE FRAMEWORKS

Table 7.1: Inference framework

Name Github Ref.	Parallel Computation			Iteration-Sche.	Speculative Decoding	Serving Optimization		Attention Kernel		Main Features
	TP	PP	Offload			Memory	Batching	Initial	Incremental	
vLLM	✓		✓	✓	✓	paged	continuous batching	xFormers	Paged attention	• Block-level KV cache • Enlarging batch size
FlexGen*		✓	✓		×	×	×	torch. bmm	torch. bmm	• CPU&Disk Offload • Maximizing single GPU
TGI	✓			✓	✓	-	-	Flash attention	Paged attention	• Huggingface integration
DeepSpeed Inference	✓	✓			×	blocked	split-and-fuse	cuBLAS GEMM	cuBLAS GEMM	• Kernel auto-injection • Multi-GPU & Multi-Node
ZeRO Inference*	✓	✓	✓		×	×	×	cuBLAS GEMM	cuBLAS GEMM	• CPU&NVMe Offload • Maximizing single GPU
TensorRT-LLM	✓	✓	✓	✓	✓	paged	continuous batching	cuBLAS/Flash-attn	Paged attention	• NVIDIA Triton integration • Rich features supported

Generative LLM serving requires a full stack of optimizations and many recent works have started to develop software frameworks to provide efficient LLM inference deployment service. In the following, we revisit these systems and investigate a comprehensive analysis of several representative open-sourced GPU-based LLM serving systems in Table 7.1. The analysis does not contain some popular related projects, including deployment solutions built on top of the other systems, like OpenLLM (vLLM), xinference (ggml + vLLM + xFormers), LMDeploy (FasterTransformer), gpt-fast (PyTorch), DeepSpeed-MII and DeepSpeed-FastGen [20] (DeepSpeed-Inference), and RayLLM and RayServe (vLLM).

We compare these state-of-the-art LLM serving systems and summarize their differences in several aspects. First, most of these systems support tensor parallelism to enable multi-GPU inference and improve the system performance. And some of them future support pipeline parallelism or offloading to support inference over multi-node or resource-constrained environments individually. Second, partial systems learn from Orca and implement the iteration-level scheduling. Third, we investigate the attention kernels of these systems and introduce their implementations in terms of the initial and incremental phases respectively.

For the initial phase, they usually adapt a batched general matrix multiply (GEMM) approach (e.g., cuBLAS, torch, Relay) and some utilize the online softmax trick to reduce HBM access (e.g., Flash-attention, xFormers). The incremental phase is more challenging because the per-token generation scheme results in lower computational intensity. To improve the GPU utilization, FasterTransformer manually fuses the attention calculations (e.g., linear projection, positional bias, dot product, softmax, etc) into a single high-performance kernel template and involves several kernel optimization techniques, such as caching with shard memory, warp-shuffle instruction for reduction, half matrix multiplication and accumulation (HMMA) with tensor core and multiple-precision support. FlexFlow-Serve enables speculative decoding and provides a tree-based parallel decoding kernel to verify the speculated tokens from multiple sequences (i.e., from multiple small models or different beams or parallel sampling) with zero-memory redundancy and maximum thread parallelism. vLLM extends the fused multi-head attention (MHA) kernel from FasterTransformer by partitioning the KV cache into pages to eliminate redundant memory usage, especially for parallel sampling scenarios.

Note that, there still remain some other notable aspects that are not covered by the above discussions. For example, even for the most popular Flash and Paged attention kernels, they are usually implemented in different ways across these systems. TGI directly imports the original Flash/Paged attention libraries, and TensorRT-LLM modifies from FasterTransformer’s fused attention kernel to support paged attention. Another example is about the input-aware kernel selection. For the initial phase, TensorRT-LLM selects from cuBLAS and Flash attention based on the context length.

Besides the attention calculation, for the linear projection operators, there is also a recent trend of replacing GEMM with general matrix-vector product (GEMV) to handle the cases of small batch size (i.e., 1) more efficiently. And these systems also have many other different features, such as programming language (i.e., C++, Python), low-precision support (i.e., FP16, INT8), supported hardware and models. In summary, these different choices of design and implementation are largely determined by their prioritized optimization target. For example, vLLM proposes paged attention to improve the batch size for higher *throughput*, while FlexFlow-Serve leverages SpecInfer to accelerate decoding for lower *latency*. Basically, low latency and high throughput are dual optimization targets in LLM serving systems, representing complementary but often conflicting objectives, necessitating a balanced strategy to optimize the trade-off between rapid response for individual tasks and maximizing the volume of tasks processed over a specified time frame.

Some recent studies^[14] further decompose the response latency by $TTFT + TPOT \times \text{output sequence length}$, where TTFT represents *Time To First Token* and TPOT represents *Time Per Output Token*. The former is driven by the initial phase processing speed while the latter directly depends on per-iteration execution time during incremental decoding. Distinguishing these two metrics is beneficial to LLM service providers, leading to different system design choices and user experience (e.g., faster application responsiveness^[35], longer prompts^[20]).

Besides, reducing the *monetary cost* is also an important and practical objective for the design and implementation of some LLM serving systems^[37]. Although it is unlikely to have a one-size-fits-all solution, we believe that future LLM serving systems will continually integrate these differentiated features, thereby continuously improving system efficiency and hardware utilization.

8 CONCLUSION

Efficient LLM inference focuses on reducing the computational, memory access, and memory costs during LLM inference processes, aiming to optimize efficiency metrics such as latency, throughput, storage, power, and energy. This paper offers a comprehensive review of efficient LLM inference research, presenting insights, recommendations, and future directions for key techniques. Initially, we introduce a hierarchical taxonomy encompassing model-level, and system-level optimizations. Subsequently, guided by this taxonomy, we meticulously examine and summarize studies at each level and sub-field. For well-established techniques like model quantization and efficient serving systems, we conduct experiments to evaluate and analyze their performance. Based on these analyses, we offer practical suggestions and identify promising research avenues for practitioners and researchers in the field.

References

- [1] Agrawal, A., Kedia, N., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B.S., Tumanov, A., Ramjee, R., 2024. Taming throughput-latency tradeoff in llm inference with sarathi-serve .
- [2] Agrawal, A., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B.S., Ramjee, R., 2023. Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. URL: <https://arxiv.org/abs/2308.16369>, arXiv:2308.16369.
- [3] Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., Sanghai, S., 2023. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. URL: <https://arxiv.org/abs/2305.13245>, arXiv:2305.13245.
- [4] Aminabadi, R.Y., Rajbhandari, S., Zhang, M., Awan, A.A., Li, C., Li, D., Zheng, E., Rasley, J., Smith, S., Ruwase, O., He, Y., 2022. Deepspeed inference: Enabling efficient inference of transformer models at unprecedented scale. URL: <https://arxiv.org/abs/2207.00032>, arXiv:2207.00032.
- [5] Anagnostidis, S., Pavlo, D., Biggio, L., Noci, L., Lucchi, A., Hofmann, T., 2024. Dynamic context pruning for efficient and interpretable autoregressive transformers. URL: <https://arxiv.org/abs/2305.15805>, arXiv:2305.15805.
- [6] Beltagy, I., Peters, M.E., Cohan, A., 2020. Longformer: The long-document transformer. URL: <https://arxiv.org/abs/2004.05150>, arXiv:2004.05150.
- [7] Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J.D., Chen, D., Dao, T., 2024. Medusa: Simple llm inference acceleration framework with multiple decoding heads. URL: <https://arxiv.org/abs/2401.10774>, arXiv:2401.10774.
- [8] Chen, C., Borgeaud, S., Irving, G., Lespiau, J.B., Sifre, L., Jumper, J., 2023. Accelerating large language model decoding with speculative sampling. URL: <https://arxiv.org/abs/2302.01318>, arXiv:2302.01318.
- [9] Child, R., Gray, S., Radford, A., Sutskever, I., 2019. Generating long sequences with sparse transformers. URL: <https://arxiv.org/abs/1904.10509>, arXiv:1904.10509.
- [10] CMU, Stanford, UCSD, . Flexflow-serve. URL: <https://github.com/Flexflow/FlexFlow/tree/inference>.
- [11] Dao, T., 2023. Flashattention-2: Faster attention with better parallelism and work partitioning. URL: <https://arxiv.org/abs/2307.08691>, arXiv:2307.08691.
- [12] Dao, T., Fu, D.Y., Ermon, S., Rudra, A., Ré, C., 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. URL: <https://arxiv.org/abs/2205.14135>, arXiv:2205.14135.
- [13] Dao, T., Haziza, D., Massa, F., Sizov, G., . Flash-decoding for long-context inference. URL: <https://crfm.stanford.edu/2023/10/12/flashdecoding.html>.
- [14] Databricks, . Llm inference performance engineering: Best practices. URL: <https://www.databricks.com/blog/llminference-performance-engineering-best-practices>.
- [15] Dettmers, T., Lewis, M., Belkada, Y., Zettlemoyer, L., 2022. Llm.int8(): 8-bit matrix multiplication for transformers at scale. URL: <https://arxiv.org/abs/2208.07339>, arXiv:2208.07339.
- [16] Frantar, E., Ashkboos, S., Hoefler, T., Alistarh, D., 2023. Gptq: Accurate post-training quantization for generative pre-trained transformers. URL: <https://arxiv.org/abs/2210.17323>, arXiv:2210.17323.

REFERENCES

- [17] Fu, Y., Bailis, P., Stoica, I., Zhang, H., 2024. Break the sequential dependency of llm inference using lookahead decoding. URL: <https://arxiv.org/abs/2402.02057>, arXiv:2402.02057.
- [18] Gim, I., Chen, G., seob Lee, S., Sarda, N., Khandelwal, A., Zhong, L., 2024. Prompt cache: Modular attention reuse for low-latency inference. URL: <https://arxiv.org/abs/2311.04934>, arXiv:2311.04934.
- [19] He, Z., Zhong, Z., Cai, T., Lee, J.D., He, D., 2024. Rest: Retrieval-based speculative decoding. URL: <https://arxiv.org/abs/2311.08252>, arXiv:2311.08252.
- [20] Holmes, C., Tanaka, M., Wyatt, M., Awan, A.A., Rasley, J., Rajbhandari, S., Aminabadi, R.Y., Qin, H., Bakhtiari, A., Kurilenko, L., He, Y., 2024. Deepspeed-fastgen: High-throughput text generation for llms via mii and deepspeed-inference. URL: <https://arxiv.org/abs/2401.08671>, arXiv:2401.08671.
- [21] Hong, K., Dai, G., Xu, J., Mao, Q., Li, X., Liu, J., Chen, K., Dong, Y., Wang, Y., 2024. Flashdecoding++: Faster large language model inference on gpus. URL: <https://arxiv.org/abs/2311.01282>, arXiv:2311.01282.
- [22] Hu, C., Huang, H., Xu, L., Chen, X., Xu, J., Chen, S., Feng, H., Wang, C., Wang, S., Bao, Y., Sun, N., Shan, Y., 2024. Inference without interference: Disaggregate llm inference for mixed downstream workloads. URL: <https://arxiv.org/abs/2401.11181>, arXiv:2401.11181.
- [23] Jiang, H., Wu, Q., Luo, X., Li, D., Lin, C.Y., Yang, Y., Qiu, L., 2024. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression. URL: <https://arxiv.org/abs/2310.06839>, arXiv:2310.06839.
- [24] Kitaev, N., Łukasz Kaiser, Levskaya, A., 2020. Reformer: The efficient transformer. URL: <https://arxiv.org/abs/2001.04451>, arXiv:2001.04451.
- [25] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C.H., Gonzalez, J.E., Zhang, H., Stoica, I., 2023. Efficient memory management for large language model serving with pagedattention, in: Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles.
- [26] Leviathan, Y., Kalman, M., Matias, Y., 2023. Fast inference from transformers via speculative decoding. URL: <https://arxiv.org/abs/2211.17192>, arXiv:2211.17192.
- [27] Li, Y., Wei, F., Zhang, C., Zhang, H., 2024a. Eagle-2: Faster inference of language models with dynamic draft trees. URL: <https://arxiv.org/abs/2406.16858>, arXiv:2406.16858.
- [28] Li, Y., Wei, F., Zhang, C., Zhang, H., 2024b. Eagle: Speculative sampling requires rethinking feature uncertainty. URL: <https://arxiv.org/abs/2401.15077>, arXiv:2401.15077.
- [29] Li, Z., Zheng, L., Zhong, Y., Liu, V., Sheng, Y., Jin, X., Huang, Y., Chen, Z., Zhang, H., Gonzalez, J.E., Stoica, I., 2023. Alpaserve: Statistical multiplexing with model parallelism for deep learning serving. URL: <https://arxiv.org/abs/2302.11665>, arXiv:2302.11665.
- [30] Lin, B., Zhang, C., Peng, T., Zhao, H., Xiao, W., Sun, M., Liu, A., Zhang, Z., Li, L., Qiu, X., Li, S., Ji, Z., Xie, T., Li, Y., Lin, W., 2024a. Infinite-llm: Efficient llm service for long context with distattention and distributed kvcache. URL: <https://arxiv.org/abs/2401.02669>, arXiv:2401.02669.
- [31] Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.M., Wang, W.C., Xiao, G., Dang, X., Gan, C., Han, S., 2024b. Awq: Activation-aware weight quantization for llm compression and acceleration. URL: <https://arxiv.org/abs/2306.00978>, arXiv:2306.00978.

REFERENCES

- [32] Liu, H., Zaharia, M., Abbeel, P., 2023a. Ring attention with blockwise transformers for near-infinite context. URL: <https://arxiv.org/abs/2310.01889>, arXiv:2310.01889.
- [33] Liu, N.F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., Liang, P., 2023b. Lost in the middle: How language models use long contexts. URL: <https://arxiv.org/abs/2307.03172>, arXiv:2307.03172.
- [34] Liu, S.y., Liu, Z., Huang, X., Dong, P., Cheng, K.T., 2023c. Llm-fp4: 4-bit floating-point quantized transformers, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics. p. 592–605. URL: <http://dx.doi.org/10.18653/v1/2023.emnlp-main.39>, doi:10.18653/v1/2023.emnlp-main.39.
- [35] Liu, Y., Li, H., Cheng, Y., Ray, S., Huang, Y., Zhang, Q., Du, K., Yao, J., Lu, S., Ananthanarayanan, G., Maire, M., Hoffmann, H., Holtzman, A., Jiang, J., 2024. Cachegen: Kv cache compression and streaming for fast large language model serving. URL: <https://arxiv.org/abs/2310.07240>, arXiv:2310.07240.
- [36] Miao, X., Oliaro, G., Zhang, Z., Cheng, X., Wang, Z., Zhang, Z., Wong, R.Y.Y., Zhu, A., Yang, L., Shi, X., Shi, C., Chen, Z., Arfeen, D., Abhyankar, R., Jia, Z., 2024. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification, in: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ACM. p. 932–949. URL: <http://dx.doi.org/10.1145/3620666.3651335>, doi:10.1145/3620666.3651335.
- [37] Miao, X., Shi, C., Duan, J., Xi, X., Lin, D., Cui, B., Jia, Z., 2023. Spotserve: Serving generative large language models on preemptible instances. URL: <https://arxiv.org/abs/2311.15566>, arXiv:2311.15566.
- [38] Miao, X., Wang, Y., Jiang, Y., Shi, C., Nie, X., Zhang, H., Cui, B., 2022. Galvatron: Efficient transformer training over multiple gpus using automatic parallelism. Proceedings of the VLDB Endowment 16, 470–479. URL: <http://dx.doi.org/10.14778/3570690.3570697>, doi:10.14778/3570690.3570697.
- [39] ModelTC, . Lightllm. URL: <https://github.com/ModelTC/lightllm/>.
- [40] NVIDIA, a. cublas: Basic linear algebra on nvidia gpus. URL: <https://developer.nvidia.com/cublas>.
- [41] NVIDIA, b. Cutlass: Cuda templates for linear algebra subroutines. URL: <https://github.com/NVIDIA/cutlass>.
- [42] NVIDIA, c. Nvidia h100 tensor core gpu architecture. URL: <https://resources.nvidia.com/en-us-tensor-core/gtc22whitepaper-hopper>.
- [43] Patel, P., Choukse, E., Zhang, C., Shah, A., Íñigo Goiri, Maleki, S., Bianchini, R., 2024. Splitwise: Efficient generative llm inference using phase splitting. URL: <https://arxiv.org/abs/2311.18677>, arXiv:2311.18677.
- [44] Pope, R., Douglas, S., Chowdhery, A., Devlin, J., Bradbury, J., Levskaya, A., Heek, J., Xiao, K., Agrawal, S., Dean, J., 2022. Efficiently scaling transformer inference. URL: <https://arxiv.org/abs/2211.05102>, arXiv:2211.05102.
- [45] Rasley, J., Rajbhandari, S., Ruwase, O., He, Y., 2020. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters, in: KDD ’20: The 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining.
- [46] Roy, A., Saffar, M., Vaswani, A., Grangier, D., 2020. Efficient content-based sparse attention with routing transformers. URL: <https://arxiv.org/abs/2003.05997>, arXiv:2003.05997.

REFERENCES

- [47] Shazeer, N., 2019. Fast transformer decoding: One write-head is all you need. URL: <https://arxiv.org/abs/1911.02150>, arXiv:1911.02150.
- [48] Sheng, Y., Zheng, L., Yuan, B., Li, Z., Ryabinin, M., Fu, D.Y., Xie, Z., Chen, B., Barrett, C., Gonzalez, J.E., Liang, P., Ré, C., Stoica, I., Zhang, C., 2023. Flexgen: High-throughput generative inference of large language models with a single gpu. URL: <https://arxiv.org/abs/2303.06865>, arXiv:2303.06865.
- [49] Shoenybi, M., Patwary, M., Puri, R., LeGresley, P., Casper, J., Catanzaro, B., 2020. Megatron-lm: Training multi-billion parameter language models using model parallelism. URL: <https://arxiv.org/abs/1909.08053>, arXiv:1909.08053.
- [50] Stern, M., Shazeer, N., Uszkoreit, J., 2018. Blockwise parallel decoding for deep autoregressive models. URL: <https://arxiv.org/abs/1811.03115>, arXiv:1811.03115.
- [51] Sun, B., Huang, Z., Zhao, H., Xiao, W., Zhang, X., Li, Y., Lin, W., 2024a. Llmunix: Dynamic scheduling for large language model serving. URL: <https://arxiv.org/abs/2406.03243>, arXiv:2406.03243.
- [52] Sun, Z., Suresh, A.T., Ro, J.H., Beirami, A., Jain, H., Yu, F., 2024b. Spectr: Fast speculative decoding via optimal transport. URL: <https://arxiv.org/abs/2310.15141>, arXiv:2310.15141.
- [53] Vaidya, N., Oh, F., Comly, N., . Optimizing inference on large language models with nvidia tensorrt-llm, now publicly available. URL: <https://github.com/NVIDIA/TensorRT-LLM>.
- [54] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I., 2023. Attention is all you need. URL: <https://arxiv.org/abs/1706.03762>, arXiv:1706.03762.
- [55] Wu, B., Liu, S., Zhong, Y., Sun, P., Liu, X., Jin, X., 2024. Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism. URL: <https://arxiv.org/abs/2404.09526>, arXiv:2404.09526.
- [56] Xia, H., Yang, Z., Dong, Q., Wang, P., Li, Y., Ge, T., Liu, T., Li, W., Sui, Z., 2024a. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. URL: <https://arxiv.org/abs/2401.07851>, arXiv:2401.07851.
- [57] Xia, H., Zheng, Z., Wu, X., Chen, S., Yao, Z., Youn, S., Bakhtiari, A., Wyatt, M., Zhuang, D., Zhou, Z., Ruwase, O., He, Y., Song, S.L., 2024b. Fp6-llm: Efficiently serving large language models through fp6-centric algorithm-system co-design. URL: <https://arxiv.org/abs/2401.14112>, arXiv:2401.14112.
- [58] Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., Han, S., 2024a. Smoothquant: Accurate and efficient post-training quantization for large language models. URL: <https://arxiv.org/abs/2211.10438>, arXiv:2211.10438.
- [59] Xiao, G., Tian, Y., Chen, B., Han, S., Lewis, M., 2024b. Efficient streaming language models with attention sinks. URL: <https://arxiv.org/abs/2309.17453>, arXiv:2309.17453.
- [60] Xu, P., Ping, W., Wu, X., McAfee, L., Zhu, C., Liu, Z., Subramanian, S., Bakhturina, E., Shoenybi, M., Catanzaro, B., 2024. Retrieval meets long context large language models. URL: <https://arxiv.org/abs/2310.03025>, arXiv:2310.03025.
- [61] Yu, G.I., Jeong, J.S., Kim, G.W., Kim, S., Chun, B.G., 2022. Orca: A distributed serving system for Transformer-Based generative models, in: 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22), USENIX Association, Carlsbad, CA. pp. 521–538. URL: <https://www.usenix.org/conference/osdi22/presentation/yu>.

REFERENCES

- [62] Yuan, Z., Shang, Y., Zhou, Y., Dong, Z., Zhou, Z., Xue, C., Wu, B., Li, Z., Gu, Q., Lee, Y.J., Yan, Y., Chen, B., Sun, G., Keutzer, K., 2024. Llm inference unveiled: Survey and roofline model insights. URL: <https://arxiv.org/abs/2402.16363>, arXiv:2402.16363.
- [63] Zaheer, M., Guruganesh, G., Dubey, A., Ainslie, J., Alberti, C., Ontanon, S., Pham, P., Ravula, A., Wang, Q., Yang, L., Ahmed, A., 2021. Big bird: Transformers for longer sequences. URL: <https://arxiv.org/abs/2007.14062>, arXiv:2007.14062.
- [64] Zhai, Y., Jiang, C., Wang, L., Jia, X., Zhang, S., Chen, Z., Liu, X., Zhu, Y., 2023. Bytetransformer: A high-performance transformer boosted for variable-length inputs. URL: <https://arxiv.org/abs/2210.03052>, arXiv:2210.03052.
- [65] Zhang, C., Liu, Z., Song, D., 2024. Beyond the speculative game: A survey of speculative execution in large language models. URL: <https://arxiv.org/abs/2404.14897>, arXiv:2404.14897.
- [66] Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C., Wang, Z., Chen, B., 2023. H₂O: Heavy-hitter oracle for efficient generative inference of large language models. URL: <https://arxiv.org/abs/2306.14048>, arXiv:2306.14048.
- [67] Zheng, L., Li, Z., Zhang, H., Zhuang, Y., Chen, Z., Huang, Y., Wang, Y., Xu, Y., Zhuo, D., Xing, E.P., Gonzalez, J.E., Stoica, I., 2022. Alpha: Automating inter- and intra-operator parallelism for distributed deep learning. URL: <https://arxiv.org/abs/2201.12023>, arXiv:2201.12023.
- [68] Zhong, Y., Liu, S., Chen, J., Hu, J., Zhu, Y., Liu, X., Jin, X., Zhang, H., 2024. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving. URL: <https://arxiv.org/abs/2401.09670>, arXiv:2401.09670.
- [69] Zhou, Y., Lyu, K., Rawat, A.S., Menon, A.K., Rostamizadeh, A., Kumar, S., Kagy, J.F., Agarwal, R., 2024. Distill-spec: Improving speculative decoding via knowledge distillation. URL: <https://arxiv.org/abs/2310.08461>, arXiv:2310.08461.